Lecture 33 — Game Theory: Win/Lose States, Nim, Sprague-Grundy

> Year Curriculum · Prerequisites: Week 9 (XOR and bit tricks), Week 14 (DP), Week 21 (DAGs, topological order) · Estimated time: 2.5 hours

1. Overview

Game problems describe two players alternating moves and ask who wins under optimal play. They look intimidating because "optimal play" seems to require reasoning about every possible strategy, but for the large class of **impartial games** there is a complete theory:

1. Every position is either **winning** or **losing** for the player to move, computable by a simple DP.
2. **Nim** is solved by XOR (Bouton's theorem).
3. The **Sprague-Grundy theorem** says every impartial game is equivalent to a Nim heap, so a sum of independent games is solved by XOR-ing **Grundy numbers**.

We also cover common disguises (staircase Nim, splitting games), games with cycles, and the most practical habit of all: **brute-force small cases, print the table, find the pattern, verify**.

2. The setting

We study games that are:

- **two-player, alternating**, with **perfect information** and **no randomness**;
- **impartial**: the set of moves available depends only on the position, not on whose turn it is (unlike chess, where each player moves only their own pieces);
- **finite**: every play ends after finitely many moves (Section 6 relaxes this);
- under **normal play**: the player who cannot move **loses**. (In **misère** play the player who cannot move wins; see Section 4.4.)

3. Winning and losing positions

Call a position a **P-position** if the **P**revious player (the one who just moved) wins, i.e. the player to move loses with optimal play, and an **N-position** if the **N**ext player (the one to move) wins. They are characterized by:

1. Every terminal position is P (the player to move cannot move and loses).
2. A position is N iff **at least one** move leads to a P-position (move there and hand your opponent a lost position).
3. A position is P iff **every** move leads to an N-position.

Because the game is finite, the "leads to" relation is a DAG and we can evaluate positions in reverse topological order — usually just increasing order of some size parameter.

3.1 Example: a subtraction game

CSES 1729 (Stick Game): a heap of n sticks, and a set of allowed removal sizes p_1, ..., p_k. For moves {1, 3, 4}:

n	0	1	2	3	4	5	6	7	8	9	10
result	L	W	L	W	W	W	W	L	W	L	W

For instance n = 7: moves lead to 6, 4, 3, all W, so 7 is L. The DP is O(n·k).

3.2 Look for patterns

If n can be 10^{18} the DP is impossible, but subtraction games with a finite move set are always eventually periodic (the state of the last max(p_i) values determines the future). For {1, 3, 4} the losing positions are exactly n mod 7 ∈ {0, 2}; for "remove 1..k" they are $n \equiv 0 \pmod{k+1}$. The workflow is:

1. write the brute-force DP;
2. print the results for n up to a few hundred;
3. guess the period or formula;
4. verify the guess against the brute force far beyond what you looked at, and if possible prove it.

4. Nim

4.1 The game and Bouton's theorem

There are n heaps of sizes a_1, ..., a_n. A move chooses one heap and removes any positive number of stones from it. **Theorem (Bouton, 1901):** a position is P iff $a_1 \oplus a_2 \oplus \ldots \oplus a_n = 0$, where ⊕ is bitwise XOR.

4.2 Proof

Let s denote the XOR of all heap sizes. We check the three characterizing properties of the set Z = {positions with s = 0}:

- The terminal position (all heaps empty) has s = 0, so it is in Z.
- **From s = 0, every move leaves s ≠ 0.** A move changes one heap a_i to $a_i' \neq a_i$; the new XOR is $s \oplus a_i \oplus a_i' = a_i \oplus a_i' \neq 0$.
- **From s ≠ 0, some move reaches s = 0.** Let b be the highest set bit of s. Some heap a_i has bit b set (otherwise bit b of s would be 0). Then $a_i \oplus s$ has bit b cleared and the same higher bits as a_i, so $a_i \oplus s < a_i$, and reducing heap i to $a_i \oplus s$ is a legal move that makes the total XOR $s \oplus a_i \oplus (a_i \oplus s) = 0$.

So Z is exactly the set of P-positions.

4.3 Finding the winning move

Heaps (3, 4, 5): s = 011 ⊕ 100 ⊕ 101 = 010. The highest bit of s is bit 1; heap 3 = 011 has it, so reduce it to 3 ⊕ 2 = 1. Check: 1 ⊕ 4 ⊕ 5 = 0. In code: for each i, if `(a[i] ^ s) < a[i]`, reducing heap i to `a[i] ^ s` is a winning move.

4.4 Misère Nim

If taking the last stone *loses*: when at least one heap has size ≥ 2, the first player wins iff the XOR is non-zero (exactly as in normal Nim); when all heaps have size ≤ 1, the first player wins iff the number of non-empty heaps is **even**. The winning strategy plays normal Nim until the move that would leave only heaps of size ≤ 1, and then leaves an odd number of size-1 heaps instead.

5. Sprague-Grundy theory

5.1 Sums of games

The **sum** $G_1 + G_2 + \ldots + G_k$ of games is played by choosing, on each turn, one component and making a move in it; a player who cannot move in any component loses. Nim is the sum of single-heap games. Knowing only whether each component is winning is not enough to decide the sum: heaps (1, 1) form a losing position but heaps (1, 2) a winning one, even though every single non-empty heap is winning. We need a finer invariant.

5.2 mex and Grundy numbers

For a set S of non-negative integers, **mex(S)** (minimum excludant) is the smallest non-negative integer not in S: mex{0, 1, 3} = 2, mex{1, 2} = 0, mex{} = 0. The **Grundy number** of a position is

g(x) = mex { g(y) : y reachable from x in one move }.

So terminal positions have g = 0, and **g(x) = 0 iff x is a P-position** (g(x) = 0 means no move reaches a position with g = 0; g(x) > 0 means some move does). For a single Nim heap of size n, g(n) = n.

5.3 The theorem

Sprague-Grundy theorem: $g(G_1 + \ldots + G_k) = g(G_1) \oplus \ldots \oplus g(G_k)$. Consequently, every impartial game is equivalent to a single Nim heap of size g, and a sum is losing iff the XOR of the components' Grundy numbers is 0.

Why it works: a position with Grundy value g can move to positions of every value 0, 1, …, g − 1 (by definition of mex), just like a Nim heap of size g. It may also be able to move to positions with values greater than g, but whatever such a move does, the opponent can answer by moving that component back down to value g (again by mex), so those moves never help. Then the Nim proof from Section 4.2 applies verbatim to the Grundy values.

5.4 Computing Grundy numbers

Iterate over positions in reverse topological order; collect the successors' values into a boolean array and take the smallest absent value. Since g(x) ≤ (number of moves from x), the array can be sized by the number of moves. For a subtraction set with k moves over n heap sizes, this is O(n·k).

Example: removing 1, 2 or 3 stones gives g(n) = n mod 4. So for several heaps with this rule (CSES 1098 Nim Game II) the first player wins iff $\oplus\ (a_i \bmod 4) \neq 0$. Note how different this is from "is any single heap winning".

5.5 Splitting games

If a move replaces a heap by two (or more) independent heaps, the resulting position is itself a sum, so its Grundy value is the XOR of the parts. **Grundy's game** (CSES 2207): a move splits a heap into two non-empty heaps of **different** sizes. Then

g(n) = mex { g(a) ⊕ g(n − a) : 1 ≤ a, a < n − a }.

Computing all values up to N costs $O(N^2)$. The P-positions ($g = 0$) are 0, 1, 2, 4, 7, 10, 20, 23, 26, 50, 53, 270, …, 932, 1222, and computation (ours went to 6000; published computations go vastly further) finds no P-position beyond 1222. For CSES 2207 with $n \le 10^6$ one therefore precomputes up to ~1300 and answers "first" for larger n. This "compute, observe, verify" approach is typical of game problems with huge limits.

Other splitting examples: Lasker's Nim (remove stones or split a heap into two), Kayles (knock down one or two adjacent pins from a row — the row splits into two rows), and CF 603C Lieges of Legendre (remove one cow, or replace a heap of size 2x by k heaps of size x).

5.6 Staircase Nim

CSES 1099 (Stair Game): stairs 1..n, stair k holds p_k balls; a move takes any positive number of balls from a stair $k \ge 2$ and puts them on stair $k - 1$. The last player to move wins. Claim: this is exactly Nim on the **even-numbered** stairs.

- Moving balls from an even stair to an odd stair removes them from the "even" Nim heaps — a Nim move.
- Moving balls from an odd stair $k \ge 3$ to the even stair $k - 1$ adds balls to an even heap. The player who is winning the even-stair Nim simply answers by moving the same balls from stair $k - 1$ to stair $k - 2$ (odd), restoring the even heaps exactly. Such moves therefore cannot help the losing player, and they cannot go on forever because balls only move downwards; balls on stair 1 never move again.

So the first player wins iff $p_2 \oplus p_4 \oplus p_6 \oplus \ldots \neq 0$. The reference code verifies this against a brute force.

6. Games on graphs and with cycles

If positions are vertices of a directed graph and a move follows an edge, a DAG is handled by the DP above in reverse topological order (Week 21). If the graph has cycles, play may go on forever and some positions are **draws**. Use **retrograde analysis**, a BFS from the terminal positions backwards:

- terminal positions are losses;
- when a position is labeled **loss**, all its predecessors become **wins**;
- keep a counter of unlabeled successors for each position; when a position's counter reaches 0 and all its successors are wins, it becomes a **loss**;
- positions never labeled are **draws**.

This is $O(V + E)$ and is the standard approach for "chase" games (cop and robber) and the like.

7. Shortcut arguments

- **Symmetry (mirror) strategy.** If the second player can always answer by restoring a symmetric position, the second player wins. Example: n coins in a circle, a move removes one coin or two adjacent coins; for n ≥ 3 the second player answers the first move by splitting the remaining arc into two equal arcs, then mirrors.
- **Strategy stealing.** If having an extra move can never hurt, the second player cannot have a winning strategy (the first player could "steal" it). This proves existence, not the strategy (Chomp, Hex).

Such arguments are great for guessing; still confirm with a brute force.

8. Reference implementations

8.1 Stick game DP — CSES 1729

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;
    vector<int> p(k);
    for (int& x : p) cin >> x;
    vector<char> win(n + 1, 0);  // win[0] = 0: no move, player to move loses
    for (int i = 1; i <= n; i++)
        for (int x : p)
            if (x <= i && !win[i - x]) { win[i] = 1; break; }
    string out;
    for (int i = 1; i <= n; i++) out += win[i] ? 'W' : 'L';
    cout << out << "\n";
}
```

Sample: `10 3 / 1 3 4` → `WLWWWWLWLW`.

8.2 Nim, Nim II, Stair game in one program

The first token selects the variant; each has t test cases, as in CSES 1730, 1098 and 1099.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string variant;
```

```
    int t;
    cin >> variant >> t;
    while (t--) {
        int n;
        cin >> n;
        int x = 0;
        for (int i = 1; i <= n; i++) {
            int a;
            cin >> a;
            if (variant == "nim") x ^= a;                       // CSES 1730
            else if (variant == "nim2") x ^= a % 4;             // CSES 1098
            else if (variant == "stair" && i % 2 == 0) x ^= a;  // CSES 1099
        }
        cout << (x ? "first" : "second") << "\n";
    }
}
```

8.3 Brute-force verification of the Nim, misère and staircase claims

Never trust a pattern you have not checked. This program solves tiny positions by memoized search directly from the rules and compares with the formulas.

```
#include <bits/stdc++.h>
using namespace std;

map<pair<vector<int>, int>, bool> memo;  // (state, mode) -> player to move wins?

// mode 0 = normal Nim, 1 = misere Nim, 2 = staircase (state[0] is stair 1)
bool wins(const vector<int>& s, int mode) {
    auto key = make_pair(s, mode);
    auto it = memo.find(key);
    if (it != memo.end()) return it->second;
    bool anyMove = false, res = false;
    for (int i = 0; i < (int)s.size() && !res; i++) {
        if (mode == 2 && i == 0) continue;  // balls on stair 1 cannot move
        for (int take = 1; take <= s[i] && !res; take++) {
            vector<int> t = s;
            t[i] -= take;
            if (mode == 2) t[i - 1] += take;
            anyMove = true;
            if (!wins(t, mode)) res = true;
        }
    }
    if (!anyMove) res = (mode == 1);  // no move: lose (normal), win (misere)
    return memo[key] = res;
}

int main() {
    int checked = 0;
    for (int a = 0; a <= 4; a++)
        for (int b = 0; b <= 4; b++)
            for (int c = 0; c <= 4; c++) {
```

```
                vector<int> s = {a, b, c};
                int x = a ^ b ^ c, ones = (a == 1) + (b == 1) + (c == 1);
                bool big = a > 1 || b > 1 || c > 1;
                assert(wins(s, 0) == (x != 0));
                assert(wins(s, 1) == (big ? x != 0 : ones % 2 == 0));
                assert(wins(s, 2) == ((b ^ 0) != 0));  // stairs 1..3: only stair 2 is
even
                checked++;
            }
    for (int a = 0; a <= 2; a++)
        for (int b = 0; b <= 2; b++)
            for (int c = 0; c <= 2; c++)
                for (int d = 0; d <= 2; d++) {
                    vector<int> s = {a, b, c, d};  // stairs 1..4: even stairs 2 and 4
                    assert(wins(s, 2) == ((b ^ d) != 0));
                    checked++;
                }
    cout << "all " << checked << " positions agree with the formulas\n";
}
```

8.4 Grundy numbers for a splitting game — CSES 2207 Grundy's Game

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    const int LIM = 2000;  // no P-positions exist above 1222
    vector<int> g(LIM + 1, 0), seen(LIM + 2, -1);
    for (int n = 1; n <= LIM; n++) {
        for (int a = 1; 2 * a < n; a++) {
            int v = g[a] ^ g[n - a];
            if (v <= LIM) seen[v] = n;  // mark "value v reachable from n"
        }
        int m = 0;
        while (seen[m] == n) m++;
        g[n] = m;
    }
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        bool firstWins = n > LIM ? true : g[n] != 0;
        cout << (firstWins ? "first" : "second") << "\n";
    }
}
```

The `seen[v] = n` trick (timestamping) avoids clearing the array for every n.

9. Common mistakes

1. **Combining components with W/L instead of Grundy numbers.** Two winning games can sum to a losing one.
2. **Misreading the end condition.** "The player who takes the last stone wins" is normal play; "loses" is misère. Re-read before coding.
3. **Wrong base case.** In normal play, the terminal position is losing for the player to move and has g = 0.
4. **mex array too small.** Grundy values can be as large as the number of moves (Grundy's game reaches 75 before n = 6000). Size by the number of moves or use timestamps.
5. **Premature pattern guessing.** Subtraction games can have long pre-periods. Verify against brute force for hundreds or thousands of values.
6. **Treating a cyclic game as a DAG.** The recursive DP loops forever; use retrograde analysis and allow draws.
7. **Off-by-one in staircase Nim.** Which stairs count depends on where balls finally become stuck (the ground / stair 1). Verify on tiny cases.
8. **Partisan games.** If players have different moves (e.g., each player moves only their own pieces), Sprague-Grundy does not apply directly.

10. Practice problems

- CSES 1729 — Stick Game (https://cses.fi/problemset/task/1729)
- AtCoder Educational DP Contest K — Stones (https://atcoder.jp/contests/dp/tasks/dp_k)
- CSES 1730 — Nim Game I (https://cses.fi/problemset/task/1730)
- CSES 1098 — Nim Game II (https://cses.fi/problemset/task/1098)
- CSES 1099 — Stair Game (https://cses.fi/problemset/task/1099)
- CSES 2207 — Grundy's Game (https://cses.fi/problemset/task/2207)
- CSES 2208 — Another Game (https://cses.fi/problemset/task/2208) — brute force a few heaps and find the pattern
- Codeforces 768E — Game of Stones (https://codeforces.com/problemset/problem/768/E)
- Codeforces 603C — Lieges of Legendre (https://codeforces.com/problemset/problem/603/C)
- Codeforces 455B — A Lot of Games (https://codeforces.com/problemset/problem/455/B) — trie game, links to Week 29
- Codeforces 1033G — Chip Game (https://codeforces.com/problemset/problem/1033/G) — hard, stretch

11. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, chapter 25 (game theory)
- cp-algorithms: "Sprague-Grundy theorem. Nim" (https://cp-algorithms.com/game_theory/sprague-grundy-nim.html) and "Games on arbitrary graphs"

(https://cp-algorithms.com/game_theory/games_on_graphs.html)

- Thomas S. Ferguson, *Game Theory* lecture notes, Part I: Impartial Combinatorial Games (free online)
- E. Berlekamp, J. Conway, R. Guy, *Winning Ways for Your Mathematical Plays* — the classic reference, including partisan games
- OEIS A002188 — Grundy values of Grundy's game